

Raport z Eksperymentów: Laboratorium 3 – MiniZinc

Data wykonania: 5 maja 2026

Środowisko: MiniZinc, Solver: Gecode

Cel: Zastosowanie zaawansowanych technik modelowania (łamanie symetrii, ograniczenia nadmiarowe, odwrócenie ograniczeń) w optymalizacji kombinatorycznej.

Zadanie 1: Kolorowanie grafu z łamaniem symetrii

Model ustala minimalną liczbę kolorów potrzebnych do pokolorowania wierzchołków grafu, tak by żadne dwa połączone wierzchołki nie miały tego samego koloru. Zastosowano łamanie symetrii poprzez sztywne przypisanie kolorów początkowych na podstawie kliku.

Rozmiar grafu	Liczba kolorów	Czas (solveTime)	Failures (Nawroty)	Węzły przeszukiwania
N = 5	3	~0.00006 s	1	3
N = 10	3	~0.00007 s	1	8
N = 20	5	~0.00011 s	10	29
N = 30	5	~0.00027 s	26	66

Wnioski: Zastosowanie łamania symetrii sprawia, że złożoność problemu drastycznie spada. Czas obliczeń nawet dla 30 wierzchołków wynosi ułamki milisekund.

Zadanie 2: Problem wielu plecaków

Celem było zmaksymalizowanie wartości zapakowanych przedmiotów do kilku identycznych plecaków. Zastosowano łamanie symetrii przy użyciu predykatu `lex_less`, który narzuca porządek leksykograficzny na zawartość torb, co eliminuje sprawdzanie symetrycznych odpowiedników.

Wariant	Suma wartości	Czas (solveTime)	Failures	Wybrane przedmioty w plecakach
A	23	~0.00012 s	25	P1: [3, 4, 5, 6], P2: [1, 2], P3: [], P4: []
B	76	~0.00405 s	2231	P1: [6, 7, 8], P2: [3], P3: [2], P4: [1, 4, 5]

Wnioski: Wariant B był znacznie trudniejszy obliczeniowo (wymagał ponad 2200 nawrotów), co widać po czasie obliczeń. Mimo to, solver znalazł optymalne rozwiązanie bardzo szybko.

Zadanie 3: Magiczny ciąg

Model generuje ciąg, w którym wartość na pozycji `i` określa, ile razy liczba `i` występuje w całym ciągu. Zastosowano **ograniczenia nadmiarowe** (redundant constraints) – sumę elementów równą `n` oraz sumę ważoną.

Długość ciągu (n)	Czas (solveTime)	Failures (Nawroty)	Węzły przeszukiwania
10	~0.00018 s	13	31
20	~0.00072 s	28	64
50	~0.00673 s	65	138
120	~0.07392 s	193	395

Wnioski: Ograniczenia nadmiarowe pozwalają na błyskawiczne wykluczanie niemożliwych gałęzi. Dla `n=120` wynik otrzymujemy w mniej niż jedną dziesiątą sekundy przy zaledwie 193 błędnych decyzjach (failures). Wynikowy ciąg składa się niemal w całości z zer.

Zadanie 4: N-Hetmanów z odwróceniem ograniczeń

Zadanie polegało na rozstawieniu N hetmanów tak, by się nie atakowały. Zastosowano predykat `inverse`, który łączy i synchronizuje logikę tablic wierszy (`rows`) i kolumn (`cols`), co umożliwia solverowi wykluczanie błędów z dwóch perspektyw naraz.

Rozmiar szachownicy (N)	Czas (solveTime)	Failures (Nawroty)	Węzły przeszukiwania
8	~0.00021 s	22	47
16	~0.00029 s	5	20
32	~0.00048 s	8	41

Wnioski: Wprowadzenie `inverse` stabilizuje proces wyszukiwania. Dla plansz `N=16` i `N=32` liczba nawrotów (odpowiednio 5 i 8) jest wręcz absurdalnie niska w stosunku do wielkości problemu.

Zadanie 5: Stabilne pary (Algorytm Gale'a-Shapleya)

Model dopasowuje `n` mężczyzn i `n` kobiet w stabilne pary, opierając się na macierzach preferencji. Para jest stabilna, gdy nie istnieje dwójka osób, która wolałaby siebie nawzajem bardziej niż swoich obecnych partnerów. Do wyświetlania wyników wykorzystano tablice stringów z imionami.

Rozmiar (N)	Czas (solveTime)	Failures	Wygenerowane Dopasowania (Mąż -> Żona)
3	~0.01055 s	0	Adam -> Celina, Bartek -> Anna, Cezary -> Beata
4	~0.00008 s	0	Damian -> Diana, Eryk -> Gosia, Filip -> Ela, Grzegorz -> Felicja
5	~0.00007 s	0	Hubert -> Lena, Igor -> Jola, Jan -> Hanna, Kamil -> Iga, Leon -> Kinga

Wnioski: Model został sformułowany na tyle precyzyjnie (wykorzystując implikacje logiczne), że solver przy $N=3$, $N=4$ i $N=5$ zaliczył **dokładnie 0 nawrotów (failures)**. Oznacza to, że propagacja ograniczeń natychmiast poprowadziła algorytm do jedyne go prawidłowego, stabilnego rozwiązania bez konieczności zgadywania.